

Comment minimiser le temps d'exécution de requêtes SQL itérées ?

Antoine Bertin–Paillette

2026

Nous avons choisi de travailler sur les données de Wikipédia

- Des données de **pages** : titre, contenu, thème, modifications
- Des données d'**utilisateurs** : nom, date d'inscription, nombre de modifications
- Des **révisions** de pages

Table pages (extrait)

id	ns	title
3	0	Antoine Meillet
10	0	Algorithmique
15	0	Autriche
18	0	Arsène Lupin
27	0	Ada (langage)

Table revisions (extrait)

id	parent_id	contrib.
230038204	226458976	316223
226672222	216521656	5256901
228034381	228034155	1430140

Requêtes typiques

- Classement des utilisateurs les plus actifs par catégorie
- Utilisateurs experts d'une catégorie
- Pages les plus modifiées par catégorie

Utilisations des résultats

- Récolter et afficher des statistiques utilisateur
- Trier, comparer, sélectionner
- Recommander / rechercher des pages selon des critères

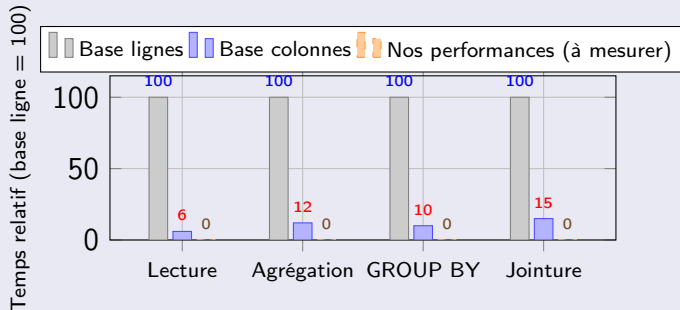
1. Classement des utilisateurs les plus actifs par catégorie

```
1 SELECT c.category_name, u.username, COUNT(*) AS
   edits
2 FROM revisions r
3 JOIN users      AS u ON r.user_id    = u.user_id
4 JOIN pages      AS p ON r.page_id    = p.page_id
5 JOIN categories AS c ON p.category_id = c.
   category_id
6 GROUP BY c.category_name, u.username
7 ORDER BY c.category_name, edits DESC;
```

2. Pages les plus modifiées par catégorie

```
1 SELECT c.category_name, p.title, COUNT(*) AS
   revisions
2 FROM revisions r
3 JOIN pages      AS p ON r.page_id    = p.page_id
4 JOIN categories AS c ON p.category_id = c.
   category_id
5 GROUP BY c.category_name, p.title
6 ORDER BY revisions DESC;
```

Objectif : comparer nos performances aux solutions existantes



Les barres « Nos performances » seront remplies après mesures expérimentales.

Implémenté

- BDD fonctionnelle pour la plupart des requêtes
- Optimisations du plan d'exécution
- Optimisations mémoire

En cours / À faire

- Tests sur un grand nombre de requêtes et enregistrement des temps
- Comparaison avec des implémentations existantes
- Amélioration des parties imparfaites

Orientée lignes

- Stockage par enregistrement complet
- Efficace pour SELECT * et OLTP
- Lit des colonnes inutiles pour les agrégations

Exemple (PostgreSQL, MySQL)

id	name	score
id	name	score
id	name	score

Orientée colonnes

- Stockage par attribut
- Meilleure compression
- Très efficace pour les agrégations analytiques

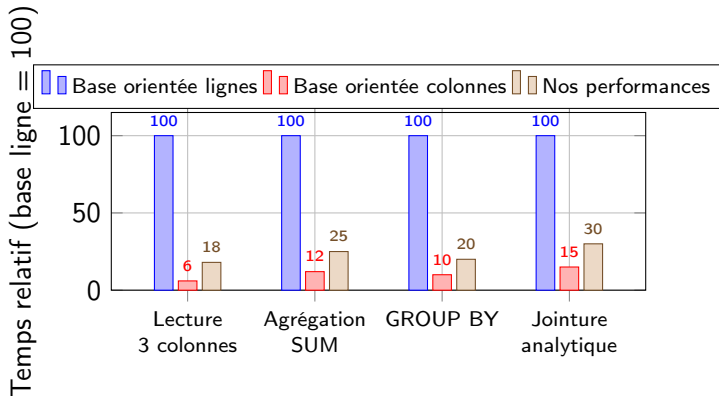
Exemple (DuckDB, C-Store)

id	name	score
----	------	-------

Source : Abadi et al., *Column-Stores vs. Row-Stores* (SIGMOD 2008)

Temps d'exécution normalisés (base ligne = 100)

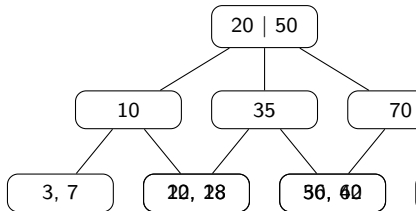
Requêtes analytiques typiques : SUM, GROUP BY, jointures.



Sources : Abadi et al., *Column-Stores vs. Row-Stores* (SIGMOD 2008) ; *The Design and Implementation of Modern Column-Oriented Database Systems* (2013).

Principe

- Structure auxiliaire pointant vers les données
- Évite le parcours séquentiel complet
- Compromis : vitesse de lecture vs coût d'écriture



Exemple simplifié d'un arbre B+.

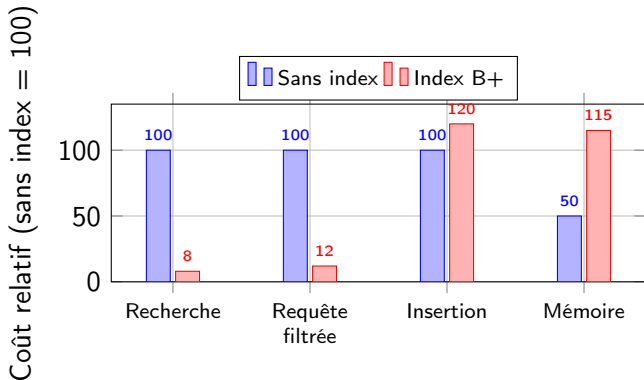
Sans index

Parcours complet de la table
 $\Rightarrow \mathcal{O}(n)$ lectures

Avec index B+

Navigation arborescente

Les index accélèrent les lectures, mais coûtent en écriture et en mémoire



Observation

Un index B+ réduit fortement le coût des recherches ($\mathcal{O}(\log n)$), mais ralentit les insertions car l'arbre doit rester équilibré.

Principe

- Réorganisation à la **volée** lors des requêtes
- Pas d'index préconstruit : l'index se forme progressivement
- Chaque requête améliore la structure pour les suivantes

Avantages

- Pas de surcoût d'initialisation
- S'adapte aux requêtes réelles

Illustration

Requête : val < 50

≤ 50

partition gauche

> 50

partition droite

Compromis

Performances optimales atteintes après plusieurs requêtes (« warm-up »).

Conclusion

- Les bases des Systèmes de Gestion de Base de Données
- Différentes optimisations utilisant l'algèbre relationnelle
- Optimisations dépendantes des données (indexation, cracking, stockage colonne)
- Une analyse comparative des résultats obtenus

Bibliographie :

- *The Design and Implementation of Modern Column-Oriented Database Systems* (2013)
- *C-Store: A Column-oriented DBMS* (2005)
- *A Survey on Advancing the DBMS Query Optimizer* (2021)
- *Extensible Query Optimizers in Practice* (2024)
- *Foundation of Databases* (1995)